

Introducing Pandas Objects

Fundamental Pandas data structures: the Series, DataFrame, and Index

```
In [1]: ▶ import numpy as np  
import pandas as pd
```

```
In [2]: ▶ #The Pandas Series Object  
data1 = pd.Series([0.25, 0.5, 0.75, 1.0])  
data1
```

```
Out[2]: 0    0.25  
1    0.50  
2    0.75  
3    1.00  
dtype: float64
```

```
In [3]: ▶ #values are simply a familiar NumPy array  
data1.values
```

```
Out[3]: array([0.25, 0.5 , 0.75, 1.  ])
```

```
In [4]: ▶ #index is an array-like object of type pd.Index  
data1.index
```

```
Out[4]: RangeIndex(start=0, stop=4, step=1)
```

```
In [5]: ▶ #Data can be accessed by the associated index via the familiar Python square-bracket notation  
data1[1]  
data1[1:3]
```

```
Out[5]: 1    0.50  
2    0.75  
dtype: float64
```

```
In [6]: #Series as generalized NumPy array, Series object is basically interchangeable  
#with a one-dimensional NumPy array
```

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],index=['a', 'b', 'c', 'd'])  
data
```

```
Out[6]: a    0.25  
       b    0.50  
       c    0.75  
       d    1.00  
       dtype: float64
```

```
In [7]: data['b']
```

```
Out[7]: 0.5
```

```
In [8]: #noncontiguous or nonsequential indices  
data = pd.Series([0.25, 0.5, 0.75, 1.0],index=[2, 5, 3, 7])  
data
```

```
Out[8]: 2    0.25  
       5    0.50  
       3    0.75  
       7    1.00  
       dtype: float64
```

```
In [9]: ▶ #Series as specialized dictionary
population_dict = {'California': 38332521,
                  'Texas': 26448193,
                  'New York': 19651127,
                  'Florida': 19552860,
                  'Illinois': 12882135}
population = pd.Series(population_dict)
population
```

```
Out[9]: California    38332521
Texas              26448193
New York           19651127
Florida            19552860
Illinois           12882135
dtype: int64
```

```
In [10]: ▶ #Constructing Series objects
pd.Series([2, 4, 6])
```

```
Out[10]: 0    2
         1    4
         2    6
dtype: int64
```

```
In [11]: ▶ pd.Series(5, index=[100, 200, 300])
```

```
Out[11]: 100    5
         200    5
         300    5
dtype: int64
```

```
In [12]: ▶ #The Pandas DataFrame Object
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,
            'Florida': 170312, 'Illinois': 149995}
```

```
In [13]: area = pd.Series(area_dict)
area
```

```
Out[13]: California    423967
Texas                695662
New York             141297
Florida              170312
Illinois             149995
dtype: int64
```

```
In [14]: states = pd.DataFrame({'population': population,
                                'area': area})
states
```

```
Out[14]:
```

	population	area
California	38332521	423967
Texas	26448193	695662
New York	19651127	141297
Florida	19552860	170312
Illinois	12882135	149995

```
In [15]: states.index
```

```
Out[15]: Index(['California', 'Texas', 'New York', 'Florida', 'Illinois'], dtype='object')
```

```
In [16]: states.columns
```

```
Out[16]: Index(['population', 'area'], dtype='object')
```

The Pandas Index Object

Series and DataFrame objects contain an explicit index that lets you reference and modify data. This Index object is an interesting structure in itself, and it can be thought of either as an immutable array or as an ordered set.

Index object serves as a fundamental building block for aligning data within Series and DataFrames. It represents the labels or identifiers for rows (or columns) in a pandas object.

Key Features of a pandas Index: Immutable: Once created, an Index object cannot be modified to maintain data integrity. However, you can create a new Index if needed. Alignment: Ensures automatic alignment of data during operations. Flexibility: Supports hierarchical indexing (MultiIndex), datetime indexing, and more.

```
In [17]: ▶ ind = pd.Index([2, 3, 5, 7, 11])
ind
```

```
Out[17]: Int64Index([2, 3, 5, 7, 11], dtype='int64')
```

Index as immutable array

The Index object in many ways operates like an array. For example, we can use standard Python indexing notation to retrieve values or slices:

```
In [18]: ▶ ind[1]
ind[::2]
```

```
Out[18]: Int64Index([2, 5, 11], dtype='int64')
```

```
In [19]: ▶ #Attributes of Index objects
print(ind.size, ind.shape, ind.ndim, ind.dtype)
```

```
5 (5,) 1 int64
```

```
In [20]: ▶ #Indices are immutable
ind[1] = 0
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[20], line 2
      1 #Indices are immutable
----> 2 ind[1] = 0

File ~\anaconda3\lib\site-packages\pandas\core\indexes\base.py:5302, in Index.__setitem__(self, key, value)
    5300 @final
    5301 def __setitem__(self, key, value):
-> 5302     raise TypeError("Index does not support mutable operations")

TypeError: Index does not support mutable operations
```

Index as ordered set

```
In [21]: ▶ #Pandas objects are designed to facilitate operations such as joins across datasets,
#which depend on many aspects of set arithmetic.
#The Index object follows many of the conventions used by Python's built-in set data structure,
#so that unions, intersections,
#differences, and other combinations can be computed in a familiar way:

indA = pd.Index([1, 3, 5, 7, 9])
indB = pd.Index([2, 3, 5, 7, 11])
#intersection
indA & indB
```

```
C:\Users\Deepthi\AppData\Local\Temp\ipykernel_14396\1676637178.py:10: FutureWarning: Index.__and__ operat
ing as a set operation is deprecated, in the future this will be a logical operation matching Series.__an
d__. Use index.intersection(other) instead.
    indA & indB
```

```
Out[21]: Int64Index([3, 5, 7], dtype='int64')
```

```
In [22]: indA | indB # union
```

```
C:\Users\Deepthi\AppData\Local\Temp\ipykernel_14396\4042773156.py:1: FutureWarning: Index.__or__ operating as a set operation is deprecated, in the future this will be a logical operation matching Series.__or___. Use index.union(other) instead.  
indA | indB # union
```

```
Out[22]: Int64Index([1, 2, 3, 5, 7, 9, 11], dtype='int64')
```

```
In [23]: indA ^ indB # symmetric difference
```

```
C:\Users\Deepthi\AppData\Local\Temp\ipykernel_14396\1357503912.py:1: FutureWarning: Index.__xor__ operating as a set operation is deprecated, in the future this will be a logical operation matching Series.__xor___. Use index.symmetric_difference(other) instead.  
indA ^ indB # symmetric difference
```

```
Out[23]: Int64Index([1, 2, 9, 11], dtype='int64')
```

```
In [24]: indA.intersection(indB)
```

```
Out[24]: Int64Index([3, 5, 7], dtype='int64')
```

Data Indexing and Selection

Indexing (e.g., `arr[2, 1]`), slicing (e.g., `arr[:, 1:5]`), masking (e.g., `arr[arr > 0]`), fancy indexing (e.g., `arr[0, [1, 5]]`), and combinations thereof (e.g., `arr[:, [1, 5]]`).

Data Selection in Series

Series object acts in many ways like a onedimensional NumPy array, and in many ways like a standard Python dictionary. If we

keep these two overlapping analogies in mind, it will help us to understand the patterns of data indexing and selection in these

```
In [25]: #Series as dictionary  
import pandas as pd  
data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])  
data
```

```
Out[25]: a    0.25  
        b    0.50  
        c    0.75  
        d    1.00  
        dtype: float64
```

```
In [26]: #Dictionary-like Python expressions and methods to examine the keys/indices and values:  
  
data.keys()
```

```
Out[26]: Index(['a', 'b', 'c', 'd'], dtype='object')
```

```
In [27]: list(data.items())
```

```
Out[27]: [('a', 0.25), ('b', 0.5), ('c', 0.75), ('d', 1.0)]
```

```
In [28]: 'a' in data
```

```
Out[28]: True
```

```
In [29]: #Series objects can even be modified by assigning to a new index value:  
data['e'] = 1.25  
data
```

```
Out[29]: a    0.25  
        b    0.50  
        c    0.75  
        d    1.00  
        e    1.25  
        dtype: float64
```


Series as one-dimensional array

A Series builds on this dictionary-like interface and provides array-style item selection via the same basic mechanisms as NumPy arrays—that is, slices, masking, and fancy indexing.

```
In [30]: ▶ # slicing by explicit index  
data['a':'c']
```

```
Out[30]: a    0.25  
        b    0.50  
        c    0.75  
        dtype: float64
```

```
In [31]: ▶ # slicing by implicit integer index  
data[0:2]
```

```
Out[31]: a    0.25  
        b    0.50  
        dtype: float64
```

```
In [32]: ▶ # masking  
data[(data > 0.3) & (data < 0.8)]
```

```
Out[32]: b    0.50  
        c    0.75  
        dtype: float64
```

```
In [33]: ▶ #fancy indexing  
data[['a', 'e']]
```

```
Out[33]: a    0.25  
        e    1.25  
        dtype: float64
```

Indexers: loc, iloc, and ix

These slicing and indexing conventions can be a source of confusion. For example, if your Series has an explicit integer index, an indexing operation such as `data[1]` will use the explicit indices, while a slicing operation like `data[1:3]` will use the implicit Python-style

```
In [34]: data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])
data
```

```
Out[34]: 1    a
         3    b
         5    c
         dtype: object
```

```
In [35]: #explicit index when indexing
data[1]
```

```
Out[35]: 'a'
```

```
In [36]: # implicit index when slicing
data[1:3]
```

```
Out[36]: 3    b
         5    c
         dtype: object
```

loc attribute allows indexing and slicing that always references the explicit index:

```
In [37]: data.loc[1]
```

```
Out[37]: 'a'
```

```
In [38]: data.loc[1:3]
```

```
Out[38]: 1    a
         3    b
         dtype: object
```

The `iloc` attribute allows indexing and slicing that always references the implicit Python-style index:

```
In [39]: data.iloc[1]
```

```
Out[39]: 'b'
```

```
In [40]: data.iloc[1:3]
```

```
Out[40]: 3    b  
         5    c  
         dtype: object
```

Third indexing attribute, `ix`, is a hybrid of the two, and for `Series` objects is equivalent to standard `[]`-based indexing.

Data Selection in DataFrame

`DataFrame` acts in many ways like a two-dimensional or structured array, and in other ways like a dictionary of `Series` structures sharing the same index.

DataFrame as a dictionary

The first analogy we will consider is the `DataFrame` as a dictionary of related `Series` objects. Let's return to our example of areas and populations of states:

```
In [41]: area = pd.Series({'California': 423967, 'Texas': 695662,
'New York': 141297, 'Florida': 170312,
'Illinois': 149995})
pop = pd.Series({'California': 38332521, 'Texas': 26448193,
'New York': 19651127, 'Florida': 19552860,
'Illinois': 12882135})
data = pd.DataFrame({'area':area, 'pop':pop})
data
```

```
Out[41]:
```

	area	pop
California	423967	38332521
Texas	695662	26448193
New York	141297	19651127
Florida	170312	19552860
Illinois	149995	12882135

The individual Series that make up the columns of the DataFrame can be accessed via dictionary-style indexing of the column name:

```
In [42]: data['area']
```

```
Out[42]: California    423967
Texas                695662
New York             141297
Florida              170312
Illinois             149995
Name: area, dtype: int64
```

We can use attribute-style access with column names that are strings:

In [43]: `data.area`

```
Out[43]: California    423967
         Texas         695662
         New York      141297
         Florida       170312
         Illinois      149995
         Name: area, dtype: int64
```

Add a new column:

In [44]: `data['density'] = data['pop'] / data['area']`
`data`

```
Out[44]:
```

	area	pop	density
California	423967	38332521	90.413926
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

DataFrame as two-dimensional array

As mentioned previously, we can also view the DataFrame as an enhanced twodimensional array. We can examine the raw underlying data array using the values attribute:

In [45]: `data.values`

```
Out[45]: array([[4.23967000e+05, 3.83325210e+07, 9.04139261e+01],
                [6.95662000e+05, 2.64481930e+07, 3.80187404e+01],
                [1.41297000e+05, 1.96511270e+07, 1.39076746e+02],
                [1.70312000e+05, 1.95528600e+07, 1.14806121e+02],
                [1.49995000e+05, 1.28821350e+07, 8.58837628e+01]])
```

Transpose the DataFrame to swap rows and columns:

```
In [46]: data.T
```

```
Out[46]:
```

	California	Texas	New York	Florida	Illinois
area	4.239670e+05	6.956620e+05	1.412970e+05	1.703120e+05	1.499950e+05
pop	3.833252e+07	2.644819e+07	1.965113e+07	1.955286e+07	1.288214e+07
density	9.041393e+01	3.801874e+01	1.390767e+02	1.148061e+02	8.588376e+01

passing a single index to an array accesses a row:

```
In [47]: data.values[0]
```

```
Out[47]: array([4.23967000e+05, 3.83325210e+07, 9.04139261e+01])
```

passing a single “index” to a DataFrame accesses a column:

```
In [48]: data['area']
```

```
Out[48]: California    423967
Texas                695662
New York             141297
Florida              170312
Illinois             149995
Name: area, dtype: int64
```

Pandas again uses the loc, iloc, and ix indexers mentioned earlier. Using the iloc indexer, we can index the underlying array as if it is a

```
In [49]: data.iloc[:3, :2]
```

```
Out[49]:
```

	area	pop
California	423967	38332521
Texas	695662	26448193
New York	141297	19651127

```
In [50]: data.loc[:, 'Illinois', : 'pop']
```

```
Out[50]:
```

	area	pop
California	423967	38332521
Texas	695662	26448193
New York	141297	19651127
Florida	170312	19552860
Illinois	149995	12882135

NumPy-style data access patterns can be used within these indexers.

For example, in the loc indexer we can combine masking and fancy indexing as in the following:

```
In [51]: data.loc[data.density > 100, ['pop', 'density']]
```

```
Out[51]:
```

	pop	density
New York	19651127	139.076746
Florida	19552860	114.806121

indexing conventions used to set or modify values

```
In [52]: data.iloc[0, 2] = 90  
data
```

```
Out[52]:
```

	area	pop	density
California	423967	38332521	90.000000
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

Additional indexing conventions

indexing refers to columns, slicing refers to rows

```
In [53]: data['Florida':'Illinois']
```

```
Out[53]:
```

	area	pop	density
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763

```
In [54]: #Another way  
data[1:3]
```

```
Out[54]:
```

	area	pop	density
Texas	695662	26448193	38.018740
New York	141297	19651127	139.076746


```
In [55]:  #Direct masking operations are also interpreted row-wise rather than column-wise  
data[data.density > 100]
```

Out[55]:

	area	pop	density
New York	141297	19651127	139.076746
Florida	170312	19552860	114.806121

Operations on Data in Pandas

Ufuncs: Index Preservation

```
In [1]: ▶ import pandas as pd  
import numpy as np
```

```
In [2]: ▶ rng = np.random.RandomState(42)  
ser = pd.Series(rng.randint(0, 10, 4))  
ser
```

```
Out[2]: 0    6  
1    3  
2    7  
3    4  
dtype: int32
```

```
In [3]: ▶ df = pd.DataFrame(rng.randint(0, 10, (3, 4)), columns=['A', 'B', 'C', 'D'])  
df
```

```
Out[3]:
```

	A	B	C	D
0	6	9	2	6
1	7	4	3	7
2	7	2	5	4

```
In [4]: ▶ np.exp(ser)
```

```
Out[4]: 0    403.428793  
1    20.085537  
2   1096.633158  
3    54.598150  
dtype: float64
```

```
In [5]: #For a slightly more complex calculation
np.sin(df * np.pi / 4)
```

```
Out[5]:
```

	A	B	C	D
0	-1.000000	7.071068e-01	1.000000	-1.000000e+00
1	-0.707107	1.224647e-16	0.707107	-7.071068e-01
2	-0.707107	1.000000e+00	-0.707107	1.224647e-16

UFuncs: Index Alignment in Series

For binary operations on two Series or DataFrame objects, Pandas will align indices in the process of performing the operation. This is very convenient when you are working with incomplete data.

```
In [6]: #Index alignment in Series
area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California': 423967}, name='area')
population = pd.Series({'California': 38332521, 'Texas': 26448193, 'New York': 19651127},
                        name='population')
```

```
In [7]: #Divide these to compute the population density
population / area
```

```
Out[7]: Alaska      NaN
California  90.413926
New York    NaN
Texas      38.018740
dtype: float64
```

```
In [8]: ▶ #The resulting array contains the union of indices of the two input arrays,  
#which we could determine using standard Python set arithmetic on these indices  
area.index | population.index
```

```
C:\Users\Deepthi\AppData\Local\Temp\ipykernel_11544\2388938907.py:3: FutureWarning: Index.__or__ operatin  
g as a set operation is deprecated, in the future this will be a logical operation matching Series.__or_  
_. Use index.union(other) instead.  
area.index | population.index
```

```
Out[8]: Index(['Alaska', 'California', 'New York', 'Texas'], dtype='object')
```

```
In [9]: ▶ #index matching is implemented this way for any of Python's built-in arithmetic expressions;  
#any missing values  
#are filled in with NaN by default  
A = pd.Series([2, 4, 6], index=[0, 1, 2])  
  
B = pd.Series([1, 3, 5], index=[1, 2, 3])  
  
A + B
```

```
Out[9]: 0    NaN  
1    5.0  
2    9.0  
3    NaN  
dtype: float64
```

Calling `A.add(B)` is equivalent to calling `A + B`, but allows optional explicit specification of the fill value for any elements in A or B that might be missing

```
In [10]: ▶ A.add(B, fill_value=0)
```

```
Out[10]: 0    2.0  
1    5.0  
2    9.0  
3    5.0  
dtype: float64
```

Index alignment in DataFrame

```
In [11]: ▶ A = pd.DataFrame(rng.randint(0, 20, (2, 2)), columns=list('AB'))  
A
```

```
Out[11]:
```

	A	B
0	1	11
1	5	1

```
In [12]: ▶ B = pd.DataFrame(rng.randint(0, 10, (3, 3)), columns=list('BAC'))  
B
```

```
Out[12]:
```

	B	A	C
0	4	0	9
1	5	8	0
2	9	2	6

```
In [13]: ▶ A + B
```

```
Out[13]:
```

	A	B	C
0	1.0	15.0	NaN
1	13.0	6.0	NaN
2	NaN	NaN	NaN

```
In [14]: ► fill = A.stack().mean()  
A.add(B, fill_value=fill)
```

```
Out[14]:
```

	A	B	C
0	1.0	15.0	13.5
1	13.0	6.0	4.5
2	6.5	13.5	10.5

Mapping between Python operators and Pandas methods

Python operator Pandas method(s)

- add()
- sub(), subtract()
- mul(), multiply()

/ truediv(), div(), divide()

// floordiv()

% mod()

** pow()

Ufuncs: Operations Between DataFrame and Series

```
In [15]: ► A = rng.randint(10, size=(3, 4))  
A
```

```
Out[15]: array([[3, 8, 2, 4],  
                [2, 6, 4, 8],  
                [6, 1, 3, 8]])
```

```
In [16]: ▶ A - A[0]
```

```
Out[16]: array([[ 0,  0,  0,  0],
                [-1, -2,  2,  4],
                [ 3, -7,  1,  4]])
```

```
In [17]: ▶ df = pd.DataFrame(A, columns=list('QRST'))
df
```

```
Out[17]:
```

	Q	R	S	T
0	3	8	2	4
1	2	6	4	8
2	6	1	3	8

```
In [18]: ▶ #To operate column-wise specify the axis keyword
df.subtract(df['R'], axis=0)
```

```
Out[18]:
```

	Q	R	S	T
0	-5	0	-6	-4
1	-4	0	-2	2
2	5	0	2	7

```
In [19]: ▶ df - df.iloc[0]
```

```
Out[19]:
```

	Q	R	S	T
0	0	0	0	0
1	-1	-2	2	4
2	3	-7	1	4

```
In [20]: #DataFrame/Series operations will automatically align indices between the two elements:
halfrow = df.iloc[0, ::2]
halfrow
```

```
Out[20]: Q    3
         S    2
         Name: 0, dtype: int32
```

```
In [21]: df - halfrow
```

```
Out[21]:
```

	Q	R	S	T
0	0.0	NaN	0.0	NaN
1	-1.0	NaN	2.0	NaN
2	3.0	NaN	1.0	NaN

Handling Missing Data

Missing data in general as null, NaN, or NA values

Trade-Offs in Missing Data Conventions

Two strategies: Mask(indicates missing values) - entirely separate Boolean array,
Sentinel value(indicates a missing entry) - indicating a missing floating-point value with NaN (Not a Number), a special value which is part of the IEEE floating-point specification.

Missing Data in Pandas

Pandas chose to use sentinels for missing data, and further chose to use two already-existing Python null values: the special floatingpoint NaN value, and the Python None object.


```
In [22]: ▶ import numpy as np
import pandas as pd
```

None: Pythonic missing data

```
In [23]: ▶ vals1 = np.array([1, None, 3, 4])
vals1
```

```
Out[23]: array([1, None, 3, 4], dtype=object)
```

```
In [24]: ▶ #The use of Python objects in an array also means that if you perform aggregations
#like sum() or min() across an array with a None value, you will generally get an error:
vals1.sum()
```

TypeError

Traceback (most recent call last)

Cell In[24], line 3

```
1 #The use of Python objects in an array also means that if you perform aggregations
2 #like sum() or min() across an array with a None value, you will generally get an error:
----> 3 vals1.sum()
```

File ~\anaconda3\lib\site-packages\numpy\core_methods.py:48, in _sum(a, axis, dtype, out, keepdims, initial, where)

```
46 def _sum(a, axis=None, dtype=None, out=None, keepdims=False,
47          initial=_NoValue, where=True):
----> 48     return umr_sum(a, axis, dtype, out, keepdims, initial, where)
```

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'

NaN: Missing numerical data(IEEE floating-point representation)



```
In [25]: ▶ vals2 = np.array([1, np.nan, 3, 4])  
vals2.dtype
```

```
Out[25]: dtype('float64')
```

Be aware that NaN is a bit like a data virus—

It infects any other object it touches. Regardless of the operation, the result of arithmetic with NaN will be another NaN:

```
In [26]: ▶ 1 + np.nan
```

```
Out[26]: nan
```

```
In [27]: ▶ 0 * np.nan
```

```
Out[27]: nan
```

```
In [28]: ▶ vals2.sum(), vals2.min(), vals2.max()
```

```
Out[28]: (nan, nan, nan)
```

```
In [29]: ▶ #NumPy does provide some special aggregations that will ignore these missing values:  
np.nansum(vals2), np.nanmin(vals2), np.nanmax(vals2)
```

```
Out[29]: (8.0, 1.0, 4.0)
```

NaN and None in Pandas

```
In [30]: ▶ pd.Series([1, np.nan, 2, None])
```

```
Out[30]: 0    1.0  
1    NaN  
2    2.0  
3    NaN  
dtype: float64
```

```
In [31]: x = pd.Series(range(2), dtype=int)
x
```

```
Out[31]: 0    0
         1    1
         dtype: int32
```

```
In [32]: #Pandas automatically converts the None to a NaN value.
x[0] = None
x
```

```
Out[32]: 0    NaN
         1    1.0
         dtype: float64
```

Upcasting conventions list in Pandas when NA values are introduced.

Typedclass	Conversion when storing NAs	NA sentinel value
floating	No change	np.nan
object	No change	None or np.nan
integer	Cast to float64	np.nan
boolean	Cast to object	None or np.nan

Operating on Null Values

Methods for detecting, removing, and replacing null values in Pandas data structures.

```
isnull()
    Generate a Boolean mask indicating missing values

notnull()
    Opposite of isnull()

dropna()
```

Detecting null values

Pandas data structures have two useful methods for detecting null data:

`isnull()` and `notnull()`.

Either one will return similar Boolean mask over the data.

```
In [33]: data = pd.Series([1, np.nan, 'hello', None])
```

```
In [34]: data.isnull()
```

```
Out[34]: 0    False
         1     True
         2    False
         3     True
         dtype: bool
```

```
In [35]: data[data.notnull()]
```

```
Out[35]: 0      1
         2    hello
         dtype: object
```

Dropping null values

dropna() (which removes NA values)

fillna() (which fills in NA values).

```
In [36]: ▶ #Series  
data.dropna()
```

```
Out[36]: 0      1  
        2    hello  
        dtype: object
```

```
In [37]: ▶ #DataFrame  
df = pd.DataFrame([[1, np.nan, 2], [2, 3, 5], [np.nan, 4, 6]])  
df
```

```
Out[37]:
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

```
In [38]: ▶ #We cannot drop single values from a DataFrame; we can only drop full rows or full columns.  
df.dropna()
```

```
Out[38]:
```

	0	1	2
1	2.0	3.0	5

```
In [39]: #Drop NA values along a different axis; axis=1 drops all columns containing a null value:
df.dropna(axis='columns')
```

```
Out[39]:
   2
0  2
1  5
2  6
```

```
In [40]: #Default is how='any', such that any row or column (depending on the axis keyword)
#containing a null value will be dropped.
#Specify how='all', which will only drop rows/columns that are all null values:
df[3] = np.nan
df
```

```
Out[40]:
   0  1  2  3
0  1.0 NaN 2 NaN
1  2.0 3.0 5 NaN
2  NaN 4.0 6 NaN
```

```
In [41]: df.dropna(axis='columns', how='all')
```

```
Out[41]:
   0  1  2
0  1.0 NaN 2
1  2.0 3.0 5
2  NaN 4.0 6
```

```
In [42]: #For finer-grained control, the thresh parameter Lets you specify a minimum number  
#of non-null values for the row/column to be kept:  
  
df.dropna(axis='rows', thresh=3)  
  
#first and last row have been dropped, because they contain only two nonnull values.
```

```
Out[42]:
```

	0	1	2	3
1	2.0	3.0	5	NaN

```
In [43]: #Filling null values  
#fillna() method, which returns a copy of the array with the null values replaced.  
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))  
data
```

```
Out[43]: a    1.0  
b    NaN  
c    2.0  
d    NaN  
e    3.0  
dtype: float64
```

```
In [44]: #Fill NA entries with a single value, such as zero:  
data.fillna(0)
```

```
Out[44]: a    1.0  
b    0.0  
c    2.0  
d    0.0  
e    3.0  
dtype: float64
```

```
In [45]: ▶ #Forward-fill to propagate the previous value forward:  
data.fillna(method='ffill')
```

```
Out[45]: a    1.0  
        b    1.0  
        c    2.0  
        d    2.0  
        e    3.0  
        dtype: float64
```

```
In [46]: ▶ #Back-fill to propagate the next values backward:  
data.fillna(method='bfill')
```

```
Out[46]: a    1.0  
        b    2.0  
        c    2.0  
        d    3.0  
        e    3.0  
        dtype: float64
```

```
In [47]: ▶ df
```

```
Out[47]:
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
In [48]: ▶ df.fillna(method='ffill', axis=1)
```

```
Out[48]:
```

	0	1	2	3
0	1.0	1.0	2.0	2.0
1	2.0	3.0	5.0	5.0
2	NaN	4.0	6.0	6.0


```
In [49]: ▶ df.fillna(method='ffill', axis=0)
```

```
Out[49]:
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	2.0	4.0	6	NaN

Hierarchical Indexing(multi-indexing)

data indexed by more than one or two keys in higher-dimensional data store

```
In [1]: ▶ import pandas as pd
import numpy as np
```

A Multiply Indexed Series

Represent two-dimensional data within a one-dimensional Series.

```
In [2]: ▶ #The bad way
index = [('California', 2000), ('California', 2010), ('New York', 2000), ('New York', 2010),
        ('Texas', 2000), ('Texas', 2010)]
populations = [33871648, 37253956, 18976457, 19378102, 20851820, 25145561]
pop = pd.Series(populations, index=index)
pop
```

```
Out[2]: (California, 2000)    33871648
(California, 2010)    37253956
(New York, 2000)      18976457
(New York, 2010)      19378102
(Texas, 2000)         20851820
(Texas, 2010)         25145561
dtype: int64
```

```
In [3]: ▶ #Index or slice the series based on this multiple index:
pop[['California', 2010]:('Texas', 2000)]
```

```
Out[3]: (California, 2010)    37253956
(New York, 2000)      18976457
(New York, 2010)      19378102
(Texas, 2000)         20851820
dtype: int64
```

In [4]: `#If you need to select all values from 2010--which is messy`
`pop[[i for i in pop.index if i[1] == 2010]]`

Out[4]: (California, 2010) 37253956
 (New York, 2010) 19378102
 (Texas, 2010) 25145561
 dtype: int64

In [5]: `#The better way: Pandas MultiIndex`
`#Create a multi-index from the tuples as follows:`
`index = pd.MultiIndex.from_tuples(index)`
`index`
`#MultiIndex contains multiple levels of indexing--in this case, the state`
`#names and the years, as well as multiple labels for each data point which encode these levels.`

Out[5]: MultiIndex([('California', 2000),
 ('California', 2010),
 ('New York', 2000),
 ('New York', 2010),
 ('Texas', 2000),
 ('Texas', 2010)],
)

In [6]: `#If we reindex our series with this MultiIndex, we see the hierarchical`
`#representation of the data:`
`pop = pop.reindex(index)`
`pop`

Out[6]: California 2000 33871648
 2010 37253956
 New York 2000 18976457
 2010 19378102
 Texas 2000 20851820
 2010 25145561
 dtype: int64

```
In [7]: ▶ #first two columns of the Series representation show the multiple index values,  

#while the third column shows the data.  

#Notice that some entries are missing in  

#the first column: in this multi-index representation, any blank entry indicates the  

#same value as the line above it.
```

```
In [8]: ▶ #To access all data for which the second index is 2010, we can simply use the  

#Pandas slicing notation:
```

```
pop[:, 2010]
```

```
#The result is a singly indexed array with just the keys we're interested in.
```

```
Out[8]: California    37253956  

New York      19378102  

Texas         25145561  

dtype: int64
```

MultilIndex as extra dimension

The `unstack()` method will quickly convert a multiplyindexed Series into a conventionally indexed DataFrame:

```
In [9]: ▶ pop_df = pop.unstack()  

pop_df
```

```
Out[9]:
```

	2000	2010
California	33871648	37253956
New York	18976457	19378102
Texas	20851820	25145561

In [10]: `# stack() method provides the opposite operation:
pop_df.stack()`

```
Out[10]: California  2000    33871648
          2010    37253956
          New York  2000    18976457
          2010    19378102
          Texas    2000    20851820
          2010    25145561
          dtype: int64
```

In [11]: `#to add another column of demographic data for each state at each year (population under 18);
#with a MultiIndex this is as easy as adding another column to the DataFrame:
pop_df = pd.DataFrame({'total': pop,
 'under18': [9267089, 9284094, 4687374, 4318033, 5906301, 6879014]})
pop_df`

```
Out[11]:
```

		total	under18
California	2000	33871648	9267089
	2010	37253956	9284094
New York	2000	18976457	4687374
	2010	19378102	4318033
Texas	2000	20851820	5906301
	2010	25145561	6879014

```
In [12]: ► #Compute the fraction of people under 18 by year, given the above data:
f_u18 = pop_df['under18'] / pop_df['total']

f_u18.unstack()
```

Out[12]:

	2000	2010
California	0.273594	0.249211
New York	0.247010	0.222831
Texas	0.283251	0.273568

Methods of MultiIndex Creation

```
In [13]: ► #The most straightforward way to construct a multiply indexed Series or DataFrame
#is to simply pass a list of two or more index arrays to the constructor.
df = pd.DataFrame(np.random.rand(4, 2), index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
                  columns=['data1', 'data2'])
df
```

Out[13]:

		data1	data2
a	1	0.033545	0.170313
	2	0.747008	0.980583
b	1	0.727109	0.090205
	2	0.944142	0.990257

```
In [14]: #if you pass a dictionary with appropriate tuples as keys, Pandas will automatically
#recognize this and use a MultiIndex by default:
data = {('California', 2000): 33871648,
        ('California', 2010): 37253956,
        ('Texas', 2000): 20851820,
        ('Texas', 2010): 25145561,
        ('New York', 2000): 18976457,
        ('New York', 2010): 19378102}
pd.Series(data)
```

```
Out[14]: California 2000    33871648
          2010    37253956
          Texas    2000    20851820
          2010    25145561
          New York 2000    18976457
          2010    19378102
          dtype: int64
```

MultiIndex level names

```
In [15]: #Names of the levels of the MultiIndex can be accomplished by passing the names argument
#to any of the above MultiIndex constructors,
#or by setting the names attribute of the index after the fact:

pop.index.names = ['state', 'year']
pop
```

```
Out[15]: state    year
California 2000    33871648
          2010    37253956
New York   2000    18976457
          2010    19378102
Texas      2000    20851820
          2010    25145561
          dtype: int64
```

MultiIndex for columns

In [16]: **▶** *#In a DataFrame, the rows and columns are completely symmetric, and just as the rows
#can have multiple levels of indices, the columns can have multiple levels as well. Consider
#the following, which is a mock-up of some (somewhat realistic) medical data:*

In [38]: **▶** *# hierarchical indices and columns*
 index = pd.MultiIndex.from_product([[2013, 2014], [1, 2]], names=['year', 'visit'])
 columns = pd.MultiIndex.from_product([['Bob', 'Guido', 'Sue'], ['HR', 'Temp']],
 names=['subject', 'type'])

In [18]: **▶** *# mock some data*
 data = np.round(np.random.randn(4, 6), 1)
 data[:, ::2] *= 10
 data += 37

In [19]: **▶** *# create the DataFrame*
 health_data = pd.DataFrame(data, index=index, columns=columns)
 health_data

Out[19]:

subject		Bob		Guido		Sue	
	type	HR	Temp	HR	Temp	HR	Temp
year	visit						
2013	1	40.0	37.8	58.0	35.8	44.0	35.0
	2	30.0	34.5	32.0	37.8	31.0	36.0
2014	1	18.0	38.4	35.0	37.8	52.0	36.6
	2	36.0	36.7	29.0	37.1	33.0	38.9


```
In [20]: ► #index the top-level column by the person's name and get a full DataFrame
          #containing just that person's information:

health_data['Guido']
```

Out[20]:

	type	HR	Temp
year	visit		
2013	1	58.0	35.8
	2	32.0	37.8
2014	1	35.0	37.8
	2	29.0	37.1

Indexing and Slicing a MultiIndex

Multiply indexed Series

```
In [21]: ► #Consider the multiply indexed Series of state populations we saw earlier:
pop
```

```
Out[21]: state      year
California 2000      33871648
           2010      37253956
New York   2000      18976457
           2010      19378102
Texas      2000      20851820
           2010      25145561
dtype: int64
```

```
In [22]: ► #We can access single elements by indexing with multiple terms:
pop['California', 2000]
```

Out[22]: 33871648

```
In [23]: #MultiIndex also supports partial indexing, or indexing just one of the levels in the index.  
#The result is another Series, with the lower-level indices maintained:  
pop['California']
```

```
Out[23]: year  
2000    33871648  
2010    37253956  
dtype: int64
```

```
In [24]: #Partial slicing is available as well, as long as the MultiIndex is sorted  
pop.loc['California':'New York']
```

```
Out[24]: state    year  
California 2000    33871648  
           2010    37253956  
New York   2000    18976457  
           2010    19378102  
dtype: int64
```

```
In [25]: #Other types of indexing and selection-----selection based on Boolean masks:  
pop[pop > 22000000]
```

```
Out[25]: state    year  
California 2000    33871648  
           2010    37253956  
Texas      2010    25145561  
dtype: int64
```

```
In [26]: #Selection based on fancy indexing also works:  
pop[['California', 'Texas']]
```

```
Out[26]: state    year  
California 2000    33871648  
           2010    37253956  
Texas      2000    20851820  
           2010    25145561  
dtype: int64
```

Multiply indexed DataFrames

In [27]: `#Consider our toy medical DataFrame from before:
health_data`

Out[27]:

	subject	Bob		Guido		Sue	
	type	HR	Temp	HR	Temp	HR	Temp
year	visit						
2013	1	40.0	37.8	58.0	35.8	44.0	35.0
	2	30.0	34.5	32.0	37.8	31.0	36.0
2014	1	18.0	38.4	35.0	37.8	52.0	36.6
	2	36.0	36.7	29.0	37.1	33.0	38.9

In [28]: `#columns are primary in a DataFrame, and the syntax used for multiply indexed
#Series applies to the columns.
#For example, we can recover Guido's heart rate data with a simple operation:

health_data['Guido', 'HR']`

Out[28]:

year	visit	
2013	1	58.0
	2	32.0
2014	1	35.0
	2	29.0

Name: (Guido, HR), dtype: float64

In [29]: `#with the single-index case, we can use the loc, iloc, and ix indexers`
`health_data.iloc[:2, :2]`

Out[29]:

	subject	Bob	
	type	HR	Temp
year	visit		
2013	1	40.0	37.8
	2	30.0	34.5

In [30]: `#These indexers provide an array-like view of the underlying two-dimensional data,`
`#but each individual index in loc or iloc can be passed a tuple of multiple indices.`
`#For example:`

`health_data.loc[:, ('Bob', 'HR')]`

Out[30]:

year	visit	
2013	1	40.0
	2	30.0
2014	1	18.0
	2	36.0

Name: (Bob, HR), dtype: float64

Rearranging Multi-Indices

Stacking and unstacking indices

```
In [31]: #It is possible to convert a dataset from a stacked multi-index to a simple  
#two-dimensional representation, #optionally specifying the level to use:  
  
pop.unstack(level=0)
```

Out[31]:

state	California	New York	Texas
year			
2000	33871648	18976457	20851820
2010	37253956	19378102	25145561

```
In [32]: pop.unstack(level=1)
```

Out[32]:

year	2000	2010
state		
California	33871648	37253956
New York	18976457	19378102
Texas	20851820	25145561

```
In [33]: #The opposite of unstack() is stack(), which here can be used to recover the original series:  
pop.unstack().stack()
```

Out[33]:

state	year	
California	2000	33871648
	2010	37253956
New York	2000	18976457
	2010	19378102
Texas	2000	20851820
	2010	25145561

dtype: int64

Data Aggregations on Multi-Indices

In [34]: `#Pandas has built-in data aggregation methods, such as mean(), sum(), and max().
#For hierarchically indexed data, these can be passed a level parameter that controls
#which subset of the data the aggregate is computed on.`

In [35]: `health_data`

Out[35]:

subject		Bob		Guido		Sue	
	type	HR	Temp	HR	Temp	HR	Temp
year	visit						
2013	1	40.0	37.8	58.0	35.8	44.0	35.0
	2	30.0	34.5	32.0	37.8	31.0	36.0
2014	1	18.0	38.4	35.0	37.8	52.0	36.6
	2	36.0	36.7	29.0	37.1	33.0	38.9

In [36]: `#we'd like to average out the measurements in the two visits each year. We can
#do this by naming the index level we'd like to explore, in this case the year:
data_mean = health_data.mean(level='year')
data_mean`

C:\Users\Deepthi\AppData\Local\Temp\ipykernel_22884\1394479763.py:3: FutureWarning: Using the level keyword in DataFrame and Series aggregations is deprecated and will be removed in a future version. Use groupby instead. df.median(level=1) should use df.groupby(level=1).median().
data_mean = health_data.mean(level='year')

Out[36]:

subject	Bob		Guido		Sue	
type	HR	Temp	HR	Temp	HR	Temp
year						
2013	35.0	36.15	45.0	36.80	37.5	35.50
2014	27.0	37.55	32.0	37.45	42.5	37.75

In [37]: `#Making use of the axis keyword, we can take the mean among levels on the columns as well:
data_mean.mean(axis=1, level='type')`

C:\Users\Deepthi\AppData\Local\Temp\ipykernel_22884\348801390.py:2: FutureWarning: Using the level keyword in DataFrame and Series aggregations is deprecated and will be removed in a future version. Use groupby instead. df.median(level=1) should use df.groupby(level=1).median().
data_mean.mean(axis=1, level='type')

Out[37]:

	type	HR	Temp
	year		
2013		39.166667	36.150000
2014		33.833333	37.583333

In []:

Combining Datasets : Concat, Append, Merge & Join

Concatenation of two different datasets

```
In [1]: ▶ import pandas as pd  
import numpy as np
```

```
In [2]: ▶ #create a DataFrame using functions of a particular form that will be useful below:  
def make_df(cols, ind):  
    """Quickly make a DataFrame"""  
    data = {c: [str(c) + str(i) for i in ind]  
            for c in cols}  
    return pd.DataFrame(data, ind)  
# example DataFrame  
make_df('ABC', range(3))
```

Out[2]:

	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

Recall: Concatenation of NumPy Arrays

```
In [3]: ▶ #Concatenation of Series and DataFrame objects is very similar to concatenation of  
#NumPy arrays, which can be done via the np.concatenate function
```



```
In [4]: x = [1, 2, 3]
        y = [4, 5, 6]
        z = [7, 8, 9]
        np.concatenate([x, y, z])
```

```
Out[4]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [5]: #axis keyword allows to specify the axis along which the result will be concatenated:
```

```
In [6]: x = [[1, 2], [3, 4]]
        np.concatenate([x, x], axis=1)
```

```
Out[6]: array([[1, 2, 1, 2],
               [3, 4, 3, 4]])
```

Simple Concatenation with pd.concat

```
In [7]: #pd.concat() can be used for a simple concatenation of Series or DataFrame objects
```

```
In [8]: ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
        ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
        pd.concat([ser1, ser2])
```

```
Out[8]: 1    A
        2    B
        3    C
        4    D
        5    E
        6    F
        dtype: object
```

```
In [9]: #concatenate higher-dimensional objects, such as DataFrames
```

```
In [10]: ▶ df1 = make_df('AB', [1, 2])
df2 = make_df('AB', [3, 4])
print(df1); print(df2); print(pd.concat([df1, df2]))
```

```

      A  B
1  A1  B1
2  A2  B2
      A  B
3  A3  B3
4  A4  B4
      A  B
1  A1  B1
2  A2  B2
3  A3  B3
4  A4  B4
```

```
In [11]: ▶ #pd.concat allows specification of an axis along which concatenation will take place.
#Consider the following example:
```

```
In [12]: ▶ df3 = make_df('AB', [0, 1])
df4 = make_df('CD', [0, 1])
print(df3); print(df4); print(pd.concat([df3, df4], axis=1))
```

```

      A  B
0  A0  B0
1  A1  B1
      C  D
0  C0  D0
1  C1  D1
      A  B  C  D
0  A0  B0  C0  D0
1  A1  B1  C1  D1
```

```
In [13]: ▶ #Duplicate indices
```

```
In [14]: ▶ #Important difference between np.concatenate and pd.concat is that Pandas
#concatenation preserves indices, even if the result will have duplicate indices!
```

```
In [15]: x = make_df('AB', [0, 1])
y = make_df('AB', [2, 3])
y.index = x.index # make duplicate indices!
print(x); print(y); print(pd.concat([x, y]))
```

```
      A  B
0  A0  B0
1  A1  B1
      A  B
0  A2  B2
1  A3  B3
      A  B
0  A0  B0
1  A1  B1
0  A2  B2
1  A3  B3
```

```
In [16]: #Ignoring the index.
#specify this option using the ignore_index flag set to True,
#the concatenation will create a new integer index for the resulting Series:
```

```
In [17]: print(x); print(y); print(pd.concat([x, y], ignore_index=True))
```

```
      A  B
0  A0  B0
1  A1  B1
      A  B
0  A2  B2
1  A3  B3
      A  B
0  A0  B0
1  A1  B1
2  A2  B2
3  A3  B3
```

```
In [18]: #Adding MultiIndex keys.
```

In [19]: `#Another alternative is to use the keys option to specify a label for the data sources;
#the result will be a hierarchically indexed series containing the data:`

In [20]: `print(x); print(y); print(pd.concat([x, y], keys=['x', 'y']))`

```

      A  B
0  A0  B0
1  A1  B1
      A  B
0  A2  B2
1  A3  B3
      A  B
x 0  A0  B0
  1  A1  B1
y 0  A2  B2
  1  A3  B3

```

In [21]: `#Concatenation with joins
#we mainly concatenate DataFrames with shared column names.`

In [22]: `df5 = make_df('ABC', [1, 2])
df6 = make_df('BCD', [3, 4])
print(df5); print(df6); print(pd.concat([df5, df6]))`

```

      A  B  C
1  A1  B1  C1
2  A2  B2  C2
      B  C  D
3  B3  C3  D3
4  B4  C4  D4
      A  B  C  D
1  A1  B1  C1  NaN
2  A2  B2  C2  NaN
3  NaN  B3  C3  D3
4  NaN  B4  C4  D4

```

In [23]:  *#The entries for which no data is available are filled with NA values.
#To change this, we can specify one of several options for the
#join and join_axes parameters of the concatenate function.
#By default, the join is a union of the input columns (join='outer'),
#but we can change this to an intersection of the columns using join='inner':*

In [24]:  `print(df5); print(df6);
print(pd.concat([df5, df6], join='inner'))`

	A	B	C
1	A1	B1	C1
2	A2	B2	C2
	B	C	D
3	B3	C3	D3
4	B4	C4	D4
	B	C	
1	B1	C1	
2	B2	C2	
3	B3	C3	
4	B4	C4	

In [25]:  *#The append() method
#append() method in Pandas does not modify the original object—instead,
#it creates a new object with the combined data.*

```
In [26]: ▶ print(df1); print(df2); print(df1.append(df2))
```

```
      A  B
1  A1  B1
2  A2  B2
      A  B
3  A3  B3
4  A4  B4
      A  B
1  A1  B1
2  A2  B2
3  A3  B3
4  A4  B4
```

C:\Users\Deepthi\AppData\Local\Temp\ipykernel_22964\1561387783.py:1: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.
print(df1); print(df2); print(df1.append(df2))

Combining Datasets: Merge and Join

```
In [27]: ▶ #pd.merge() function implements a number of types of joins: the one-to-one, many-to-one,
#and many-to-many joins.
#All three types of joins are accessed via an identical call to the pd.merge() interface;
#the type of join performed depends on the form of the input data.
```

One-to-one joins

```
In [28]: ▶ #column-wise concatenation
```

```
In [29]: ▶ df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],
                             'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],
                    'hire_date': [2004, 2008, 2012, 2014]})
print(df1); print(df2)
```

```
   employee    group
0      Bob  Accounting
1     Jake  Engineering
2     Lisa  Engineering
3      Sue           HR
   employee  hire_date
0     Lisa      2004
1      Bob      2008
2     Jake      2012
3     Sue      2014
```

```
In [30]: ▶ #To combine this information into a single DataFrame, we can use the pd.merge():
df3 = pd.merge(df1, df2)
df3
```

Out[30]:

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

```
In [31]: ▶ #The pd.merge() function recognizes that each DataFrame has an "employee" column,
#and automatically joins using this column as a key.
#order of entries in each column is not necessarily maintained:
#merge in general discards the index, except in the special case of merges by index
```

Many-to-one joins

In [32]: **▶** *#Many-to-one joins are joins in which one of the two key columns contains duplicate entries.
#The resulting DataFrame will preserve those duplicate entries as appropriate.*

```
df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                    'supervisor': ['Carly', 'Guido', 'Steve']})
print(df3); print(df4); print(pd.merge(df3, df4))
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

	group	supervisor
0	Accounting	Carly
1	Engineering	Guido
2	HR	Steve

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

In [33]: **▶** *#The resulting DataFrame has an additional column with the “supervisor” information,
#where the information is repeated in one or more locations as required by the inputs.*

Many-to-many joins

In [34]: **▶** *#If the key column in both the left and right array contains duplicates,
#then the result is a many-to-many merge.*


```
In [35]: ► df5 = pd.DataFrame({'group': ['Accounting', 'Accounting', 'Engineering', 'Engineering', 'HR', 'HR'],
                                'skills': ['math', 'spreadsheets', 'coding', 'linux',
                                             'spreadsheets', 'organization']})
print(df1); print(df5); print(pd.merge(df1, df5))
```

```

employee      group
0      Bob  Accounting
1      Jake  Engineering
2      Lisa  Engineering
3      Sue      HR
      group      skills
0  Accounting      math
1  Accounting  spreadsheets
2  Engineering      coding
3  Engineering      linux
4      HR  spreadsheets
5      HR  organization
employee      group      skills
0      Bob  Accounting      math
1      Bob  Accounting  spreadsheets
2      Jake  Engineering      coding
3      Jake  Engineering      linux
4      Lisa  Engineering      coding
5      Lisa  Engineering      linux
6      Sue      HR  spreadsheets
7      Sue      HR  organization
```

```
In [36]: ► #pd.merge(): it looks for one or more matching column names between the two inputs,
#and uses this as the key.
#However, often the column names will not match so nicely, and pd.merge() provides a variety
#of options for handling this.
```

```
In [37]: ► #The on keyword
#Explicitly specify the name of the key column using the on keyword,
#which takes a column name or a list of column names:
```

```
In [38]: ▶ print(df1); print(df2); print(pd.merge(df1, df2, on='employee'))
```

```
   employee   group
0      Bob  Accounting
1      Jake  Engineering
2      Lisa  Engineering
3      Sue      HR
   employee  hire_date
0      Lisa      2004
1      Bob      2008
2      Jake      2012
3      Sue      2014
   employee   group  hire_date
0      Bob  Accounting      2008
1      Jake  Engineering      2012
2      Lisa  Engineering      2004
3      Sue      HR      2014
```

```
In [39]: ▶ #The left_on and right_on keywords
#At times you may wish to merge two datasets with different column names; for example,
#we may have a dataset in which the employee name is labeled as "name" rather
#than "employee". In this case, we can use the left_on and right_on keywords
#to specify the two column names:
```

```
In [40]: ▶ df3 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],
                                'salary': [70000, 80000, 120000, 90000]})
print(df1); print(df3);
print(pd.merge(df1, df3, left_on="employee", right_on="name"))
```

```
   employee   group
0      Bob  Accounting
1      Jake  Engineering
2      Lisa  Engineering
3      Sue          HR

   name  salary
0   Bob   70000
1   Jake   80000
2   Lisa  120000
3   Sue   90000

   employee   group  name  salary
0      Bob  Accounting  Bob   70000
1      Jake  Engineering  Jake   80000
2      Lisa  Engineering  Lisa  120000
3      Sue          HR   Sue   90000
```

```
In [41]: ▶ #The result has a redundant column that we can drop if desired—for example,
#by using the drop() method of DataFrames:
```

```
In [42]: ▶ pd.merge(df1, df3, left_on="employee", right_on="name").drop('name', axis=1)
```

Out[42]:

	employee	group	salary
0	Bob	Accounting	70000
1	Jake	Engineering	80000
2	Lisa	Engineering	120000
3	Sue	HR	90000

```
In [43]: ▶ #The left_index and right_index keywords
#Rather than merging on a column, you would instead like to merge on an index.
```

```
In [44]: ▶ df1a = df1.set_index('employee')
df2a = df2.set_index('employee')
print(df1a); print(df2a)
```

```

      group
employee
Bob      Accounting
Jake      Engineering
Lisa      Engineering
Sue              HR
hire_date
employee
Lisa      2004
Bob       2008
Jake      2012
Sue       2014
```

```
In [45]: ▶ #Use the index as the key for merging by specifying the left_index and/or right_index flags
#in pd.merge():

print(df1a); print(df2a);
print(pd.merge(df1a, df2a, left_index=True, right_index=True))
```

```

      group
employee
Bob      Accounting
Jake      Engineering
Lisa      Engineering
Sue              HR
hire_date
employee
Lisa      2004
Bob       2008
Jake      2012
Sue       2014
      group  hire_date
employee
Bob      Accounting    2008
Jake      Engineering    2012
Lisa      Engineering    2004
Sue              HR      2014
```

```
In [46]: ► #DataFrames implement the join() method, which performs a merge that defaults  
#to joining on indices:
```

```
In [47]: ► print(df1a); print(df2a); print(df1a.join(df2a))
```

```
      group  
employee  
Bob      Accounting  
Jake     Engineering  
Lisa     Engineering  
Sue              HR  
      hire_date  
employee  
Lisa      2004  
Bob       2008  
Jake      2012  
Sue       2014  
      group  hire_date  
employee  
Bob      Accounting    2008  
Jake     Engineering    2012  
Lisa     Engineering    2004  
Sue              HR      2014
```

```
In [48]: #to mix indices and columns, you can combine left_index with right_on or left_on
#with right_index
#to get the desired behavior:
print(df1a); print(df3);
print(pd.merge(df1a, df3, left_index=True, right_on='name'))
```

```

                group
employee
Bob      Accounting
Jake      Engineering
Lisa      Engineering
Sue              HR
   name  salary
0  Bob   70000
1  Jake  80000
2  Lisa 120000
3  Sue   90000
   group  name  salary
0  Accounting  Bob   70000
1  Engineering Jake   80000
2  Engineering Lisa  120000
3           HR  Sue   90000

```

```
In [ ]:
```

Aggregation and Grouping

Simple Aggregation in Pandas

```
In [1]: ▶ import numpy as np  
import pandas as pd
```

```
In [2]: ▶ rng = np.random.RandomState(42)  
ser = pd.Series(rng.rand(5))  
ser
```

```
Out[2]: 0    0.374540  
1    0.950714  
2    0.731994  
3    0.598658  
4    0.156019  
dtype: float64
```

```
In [3]: ▶ ser.sum()
```

```
Out[3]: 2.811925491708157
```

```
In [4]: ▶ ser.mean()
```

```
Out[4]: 0.5623850983416314
```

```
In [5]: ▶ df = pd.DataFrame({'A': rng.rand(5), 'B': rng.rand(5)})  
df
```

Out[5]:

	A	B
0	0.155995	0.020584
1	0.058084	0.969910
2	0.866176	0.832443
3	0.601115	0.212339
4	0.708073	0.181825

```
In [6]: ▶ df.mean()
```

Out[6]: A 0.477888
B 0.443420
dtype: float64

```
In [7]: ▶ df.mean(axis='columns')
```

Out[7]: 0 0.088290
1 0.513997
2 0.849309
3 0.406727
4 0.444949
dtype: float64

Listing of Pandas aggregation methods

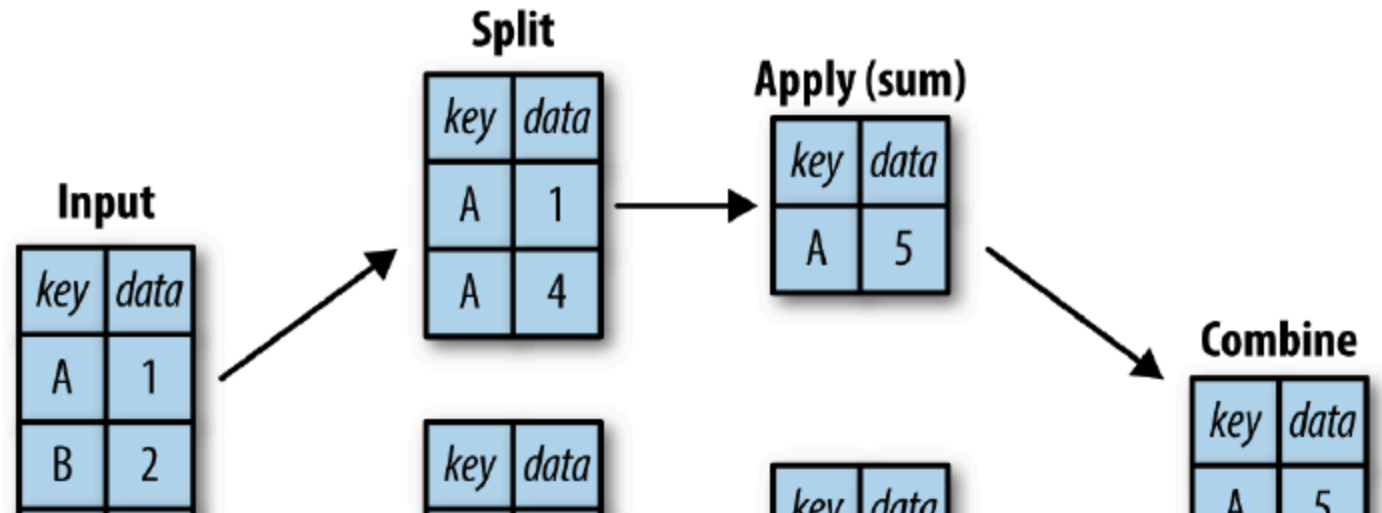
Aggregation	Description
<code>count()</code>	Total number of items
<code>first(), last()</code>	First and last item
<code>mean(), median()</code>	Mean and median
<code>min(), max()</code>	Minimum and maximum
<code>std(), var()</code>	Standard deviation and variance

GroupBy: Split, Apply, Combine

The name “group by” comes from a command in the SQL database language, but it is perhaps more illuminative to think of it in the terms first coined by Hadley Wickham of Rstats fame: split, apply, combine.

Split, apply, combine

A visual representation of a groupby operation



```
In [8]: ▶ df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'], 'data': range(6)},
                             columns=['key', 'data'])
df
```

Out[8]:

	key	data
0	A	0
1	B	1
2	C	2
3	A	3
4	B	4
5	C	5

```
In [9]: ▶ #compute the most basic split-apply-combine operation with the groupby()
          #method of DataFrames, passing the name of the desired key column:
```

```
In [10]: ▶ df.groupby('key')
```

Out[10]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001950E3035B0>

```
In [11]: ▶ #we can apply an aggregate to this DataFrameGroupBy object,  
#which will perform the appropriate apply/combine steps to produce the desired result:
```

```
In [12]: ▶ df.groupby('key').sum()
```

Out[12]:

	data
key	
A	3
B	5
C	7

The GroupBy object

The most important operations made available by a GroupBy are aggregate, filter, transform, and apply.

```
In [13]: ▶ #GroupBy objects have aggregate(), filter(), transform(), and apply() methods that efficiently  
#implement a variety of useful operations before combining the grouped data.
```

```
In [14]: ▶ rng = np.random.RandomState(0)
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                  'data1': range(6),
                  'data2': rng.randint(0, 10, 6)},
                  columns = ['key', 'data1', 'data2'])
df
```

Out[14]:

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

Aggregation.

aggregate() method takes a string, a function, or a list thereof, and compute all the aggregates at once.

```
In [15]: ▶ df.groupby('key').aggregate(['min', np.median, max])
```

Out[15]:

	data1			data2		
	min	median	max	min	median	max
key						
A	0	1.5	3	3	4.0	5
B	1	2.5	4	0	3.5	7
C	2	3.5	5	3	6.0	9

```
In [16]: ▶ #Another useful pattern is to pass a dictionary mapping column names to operations  
#to be applied on that column:
```

```
In [17]: ▶ df.groupby('key').aggregate({'data1': 'min', 'data2': 'max'})
```

Out[17]:

	data1	data2
key		
A	0	5
B	1	7
C	2	9

Filtering.

A filtering operation allows you to drop data based on the group properties. For example, we might want to keep all groups in which the standard deviation is larger than some critical value:

```
In [18]: ▶ def filter_func(x):
           return x['data2'].std() > 4
           print(df)
           print(df.groupby('key').std())
           print(df.groupby('key').filter(filter_func))
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
A	2.12132	1.414214	
B	2.12132	4.949747	
C	2.12132	4.242641	

	key	data1	data2
1	B	1	0
2	C	2	3
4	B	4	7
5	C	5	9

Transformation.

While aggregation must return a reduced version of the data, transformation can return some transformed version of the full data to recombine. For such a transformation, the output is the same shape as the input. A common example is to center the data by subtracting the group-wise mean:

```
In [19]: ▶ df.groupby('key').transform(lambda x: x - x.mean())
```

Out[19]:

	data1	data2
0	-1.5	1.0
1	-1.5	-3.5
2	-1.5	-3.0
3	1.5	-1.0
4	1.5	3.5
5	1.5	3.0

The apply() method.

The apply() method lets you apply an arbitrary function to the group results. The function should take a DataFrame, and return either a Pandas object (e.g., DataFrame, Series) or a scalar; the combine operation will be tailored to the type of output returned. For example, here is an apply() that normalizes the first column by the sum of the second:

```
In [20]: ► def norm_by_data2(x):
# x is a DataFrame of group values
    x['data1'] /= x['data2'].sum()
    return x
print(df);
print(df.groupby('key').apply(norm_by_data2))
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
0	A	0.000000	5
1	B	0.142857	0
2	C	0.166667	3
3	A	0.375000	3
4	B	0.571429	7
5	C	0.416667	9

C:\Users\Deepthi\AppData\Local\Temp\ipykernel_10496\1125903017.py:6: FutureWarning: Not prepending group keys to the result index of transform-like apply. In the future, the group keys will be included in the index, regardless of whether the applied function returns a like-indexed object.
To preserve the previous behavior, use

```
>>> .groupby(..., group_keys=False)
```

To adopt the future behavior and silence this warning, use

```
>>> .groupby(..., group_keys=True)
print(df.groupby('key').apply(norm_by_data2))
```

Specifying the split key

we split the DataFrame on a single column name. This is just one of many options by which the groups can be defined,


```
In [21]: ▶ #A list, array, series, or index providing the grouping keys.
#The key can be any series or list with a length matching that of the DataFrame.
```

```
In [22]: ▶ L = [0, 1, 0, 1, 2, 0]
print(df); print(df.groupby(L).sum())
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	data1	data2
0	7	17
1	4	3
2	4	7

C:\Users\Deepthi\AppData\Local\Temp\ipykernel_10496\3058263612.py:2: FutureWarning: The default value of numeric_only in DataFrameGroupBy.sum is deprecated. In a future version, numeric_only will default to False. Either specify numeric_only or select only columns which should be valid for the function.

```
print(df); print(df.groupby(L).sum())
```

A dictionary or series mapping index to group.

Another method is to provide a dictionary that maps index values to the group keys:

```
In [23]: ▶ df2 = df.set_index('key')
mapping = {'A': 'vowel', 'B': 'consonant', 'C': 'consonant'}
print(df2); print(df2.groupby(mapping).sum())
```

	data1	data2
key		
A	0	5
B	1	0
C	2	3
A	3	3
B	4	7
C	5	9

	data1	data2
key		
consonant	12	19
vowel	3	8

```
In [ ]: ▶
```